Course Assignment: Full-Stack "SecureNote" Application

Subject: Web Development Fundamentals & Architecture
Difficulty: Intermediate
Estimated Time: 2 Weeks

1. Objective

The goal of this assignment is to bridge the gap between theoretical web concepts and practical implementation. You will build a full-stack web application that demonstrates a clear understanding of the client-server architecture, the JavaScript execution environment, and secure communication protocols.

By completing this assignment, you will demonstrate proficiency in:

- **Frontend:** HTML5, CSS3, DOM Manipulation (Vanilla JS or React/Next.js).
- **Backend:** Node.js Runtime Environment.
- **Communication:** HTTP/HTTPS Protocols & Fetch API.
- **Configuration:** Environment Variables.
- **Theory:** JavaScript Engine vs. Runtime.

2. Scenario

You are hired to build a lightweight, secure internal tool called **"SecureNote"**. This tool allows authorized users to create, view, and delete text notes. A note consist of title and content. The application must separate concerns between the Client (Browser) and the Server (Node.js), ensuring that sensitive configuration data is never exposed to the public.

3. Technical Requirements

A. The Backend (Node.js)

You must build a RESTful API using **Node.js** (Express.js is recommended but raw HTTP module is allowed).

1. **Runtime:** The server must run on the Node.js Runtime.
2. **Environment Variables:** You must use a `.env` file to store:
   - `PORT`: The port number the server listens on.
   - `SECRET_TOKEN`: A private key used for simple authorization.
   - *Note: The `.env` file must NOT be committed to version control.*
3. **Endpoints:**

- `GET /api/notes`: Returns all notes (JSON).
- `POST /api/notes`: Creates a new note (Requires `Authorization` header with the `SECRET_TOKEN`).
- `DELETE /api/notes/:id`: Deletes a note (Requires `Authorization` header).

4. **HTTP Status Codes:** You must return appropriate status codes (e.g., `200 OK`, `201 Created`, `401 Unauthorized`, `404 Not Found`).

B. The Frontend (Client-Side)

You may choose **one** of the following paths:

- **Path A (Core):** Vanilla JavaScript, HTML, CSS.
- **Path B (Library/Framework):** React.js or Next.js with Tailwind CSS.

Requirements:

1. **DOM / Component Rendering:**
   - *Vanilla:* Use `document.querySelector`, `createElement`, and event listeners to update the UI dynamically without reloading the page.
   - *React.js/Next.js:* Use `useState`, `useEffect`, and component mapping to render data.
2. **HTTP Communication:** Use the **Fetch API** to communicate with your Node.js backend.
3. **UI:** A simple form to input notes and a list to display them.
4. **Error Handling:** If the backend returns an error (e.g., `401`), display a user-friendly message in the DOM.

C. Conceptual Report (Crucial)

Alongside the code, you must submit a **`REPORT.md`** file answering the following questions based on your implementation:

1. **JS Engine vs. Runtime:** Explain where your JavaScript code is being executed in the Frontend vs. the Backend. Mention the specific JavaScript Engine (e.g., V8) and the Runtime Environment (Browser vs. Node).
2. **DOM:** Explain how your frontend updates the screen. If using Vanilla JS, describe the DOM tree manipulation. If using React, explain the Virtual DOM concept.
3. **HTTP/HTTPS:** Describe the request/response cycle that happens when you click "Submit". What headers are sent? Why is HTTPS important for production, even if you used HTTP locally?
4. **Environment Variables:** Why did we store the `SECRET_TOKEN` in the backend `.env` file instead of the frontend code? What would happen if we put it in the frontend?

4. Suggested Project Structure

```
/secure-note-app
  ├── /backend
  │     ├── .env            (Contains PORT and SECRET_TOKEN)
  │     ├── .gitignore      (Ignores .env)
  │     ├── server.js       (Node.js entry point)
  │     └── package.json
  ├── /frontend
  │     ├── index.html      (If Vanilla)
  │     ├── style.css       (If Vanilla)
  │     ├── script.js       (If Vanilla)
  │     └── /src            (If React/Next)
  └── REPORT.md
```

5. Step-by-Step Implementation Guide

1. **Setup Backend:** Initialize a Node project. Install `express` and `dotenv`. Create the `.env` file.
2. **Create API:** Build the endpoints. Test them using Bruno, Postman or ThunderClient first to ensure logic works.
3. **Setup Frontend:** Create your UI.
4. **Connect:** Use `fetch('http://localhost:YOUR_PORT/api/notes')` to connect the two.
5. **Secure:** Implement the logic to send the `SECRET_TOKEN` in the Fetch headers.
6. **Document:** Write your `REPORT.md`.

6. Evaluation Rubric (Total: 100 Points)

Category	Criteria	Points
Backend Functionality	Node.js server runs, API endpoints work correctly, CRUD operations functional.	25
Security & Config	`.env` used correctly for secrets/port. `.env` excluded from Git. Auth header implemented.	20
Frontend Implementation	UI is responsive. Fetch API used correctly. DOM/State updates dynamically.	25
HTTP Protocol	Correct usage of HTTP verbs (GET, POST, DELETE) and Status Codes.	15
Conceptual Report	Clear, accurate explanation of JS Engine, Runtime, DOM, and Env Variables.	15

Category	Criteria	Points
Code Quality	Clean code, comments, proper folder structure, README instructions.	10

7. Bonus Challenges (Optional)

- **+10 Points:** Deploy on a **Cloud Host** provider (Vercel, Netlify, etc.) with **HTTPS** implemented and explain the process in the report.
- **+10 Points:** Add data persistence (save notes to a `.json` file or a database like SQLite) so notes survive a server restart.
- **+15 Points:** If data persistence using provided PocketHost API.
  - https://app-tracking.pockethost.io
    - **List/Search**: **`GET`** `/api/collections/notes/records`
    - **Create**: **`POST`** `/api/collections/notes/records`
    - **View**: **`GET`** `/api/collections/notes/records/:id`
    - **Update**: **`PATCH`** `/api/collections/notes/records/:id`
    - **Delete**: **`DELETE`** `/api/collections/notes/records/:id`
- **+5 Points:** Implement a "Loading State" in the UI while waiting for the Fetch request to complete.

8. Submission Guidelines

1. Upload your code to a public **GitHub Repository**.
2. Ensure your `README.md` contains instructions on how to install dependencies and run both the frontend and backend.
3. Submit the link to your repository via the learning portal.
4. **Warning:** If your `.env` file containing secrets is visible in your GitHub history, you will lose all Security & Config points.

Instructor Notes (For your reference only)

Why this assignment works:

1. **JS Engine/Runtime:** Students often confuse the language with the environment. By forcing them to write code for *both* Node and Browser, and then ask them to explain the difference in the report, you solidify this concept.
2. **Environment Variables:** This is a common security pitfall for juniors. Making it a graded requirement ensures they learn never to hardcode API keys in frontend code.
3. **Flexibility:** Allowing Vanilla JS ensures they understand the underlying DOM, while allowing React/Next.js keeps it relevant for modern hiring markets.

4. **HTTP:** By requiring specific status codes and headers, they move beyond "making it work" to "making it standard compliant."

Common Pitfalls to Watch For:

- **CORS Errors:** Students will likely forget to enable CORS in Express. This is a great teaching moment about browser security policies.
- **Hardcoded Secrets:** Check their Git history to ensure they didn't push the `.env` file and then delete it later.
- **Async/Await:** Watch for "Promise Pending" in the UI, indicating they didn't await the Fetch response properly.